TP 4 : Automates et divisions

L'intégralité du TP est à réaliser en C. N'hésitez pas à jeter un œil à votre cours si nécessaire pour les manipulations d'automates.

Automates déterministes

Pour manipuler les automates en C, on introduit les types construits suivants :

```
typedef struct liste_finaux
{
    int element;
    struct liste_finaux* suivant;
}liste;

typedef struct maillon_triplet
{
    int etat_depart;
    int lettre;
    int resultat;
    struct maillon_triplet* suivant;
}maillon;

typedef struct automate
{
    int initial;
    liste* finaux;
    maillon* fonction;
}automate;
```

On travaille ici, sur l'alphabet des entiers. Les états des états sont aussi numérotés.

Comme Σ et Q sont déjà définis, un automate est entièrement caractérisé par son état initial q_0, l'ensemble de ses états finaux F, représentés sous forme de listes chaînées et sa fonction de transition. Celle-c est donc de $\mathbb{N} \times \mathbb{N} \mapsto \mathbb{N}$. δ est aussi représentée sous la forme d'une liste chaînée de triplets (q, e, q') tels que $\delta(q, e) = q'$

Le type `liste` est une liste d'entiers pouvant servir à la fois pour représenter un mot ou pour représenter un ensemble d'états de l'automate.

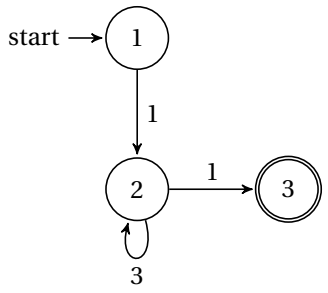

FIGURE 1 – un exemple d'automate. δ sera une liste de la forme `[(1,1,2); (2,3,2); (2, 1, 3)]`.

L'automate de la figure peut être obtenu avec les lignes de code suivant :

```
maillon m1 = {.etat_depart = 1, .lettre = 1, .resultat = 2, .suivant = NULL};
maillon m2 = {.etat_depart = 2, .lettre = 3, .resultat = 2, .suivant = &m1};
maillon m3 = {.etat_depart = 2, .lettre = 1, .resultat = 3, .suivant = &m2};
liste l = {.element = 3, .suivant = NULL};
automate a = {.initial = 1, .finaux = &l, .fonction = &m3};
```

Question 1 : Écrire une fonction `int transition(automate a, int etat, int lettre)` qui, étant donné un

automate, un état de celui-ci et une lettre renvoie l'état obtenu après transition s'il existe. Si ce n'est pas le cas, la fonction renverra -1.

Question 2 : Écrire une fonction `int delta_star(automate a, liste* mot)` qui, étant donné un automate et un mot m, renvoie $\delta^\star(q_0, m)$. On renverra -1 si la fonction n'est pas définie.

Question 3 : Écrire une fonction `bool accepte(automate a, liste* mot)` qui, étant donné un automate et un mot, décide si ce mot est accepté par l'automate. On fera attention à bien traiter les cas où la fonction n'est pas définie.

Question 4 : Écrire une fonction `automate generer_reste(int b, int d, int r)` qui étant renvoie l'automate reconnaissant exactement les mots correspondants à un entier en base b dont le reste dans la division euclidienne par d est r.

Question 5 : Adapter votre fonction `main` pour qu'elle reçoive b, d, r et une suite d'entiers strictement inférieurs à b lors de l'exécution et qu'elle affiche si cette suite représente un entier écrit en base b dont le reste de la division euclidienne par d est r. . Par exemple, si l'on exécute en ligne de commande `./a.out 7 3 0 1 1 3 3 6 2 4 1`, on teste si l'entier $1+4\times 7^1+2\times 7^2+6\times 7^3+3\times 7^4+3\times 7^5+1\times 7^6+1\times 7^7$ admet un reste nul dans la division par 3. Si vous avez des questions sur la manipulation du flux standard, posez les!

Automates non déterministes

Question 6 : Peut-on utiliser les types de la partie précédente pour définir des automates non déterministes? Créer un type `non_deterministe_automate` si nécessaire.

Question 7 : Écrire une fonction `bool accepte_nd(non_deterministe_automate a)` qui, étant donné un automate non déterministe, teste si un mot est accepté par celui-ci.

Question 8 : On veut déterminiser un automate non déterministe, comment représenter simplement une partie finie de $\mathbb{N}$ (une partie d'état de l'automate non déterministe) comme un élément de $\mathbb{N}$. Cela permet d'utiliser notre type `automate` pour déterminiser les automates.

Question 9 : Écrire une fonction `int convertir(liste* partie)` qui réalise la transformation de la question précédente.

Question 10 : Écrire une fonction `automate determiniser(non_deterministe_automate a)` qui, étant donné un automate non déterministe, renvoie un automate déterministe équivalent.

Opérations sur les automates

Question 11 : Écrire une fonction `bool est_vide(automate a)` qui, étant donné un automate détermine si celui-ci reconnaît le langage vide.

Question 12 : Écrire une fonction `non_deterministe_automate miroir(automate a)` qui étant donné un automate A renvoie l'automate (pas forcément déterministe) qui reconnaît les mots m tels que le miroir de m soit reconnu par A.

Question 13 : Modifier `main` de la question 5 pour reconnaître les nombres écrits à partir de leur bit de poids faible.